V06 - Jatayu: Echoes of the Fallen

Flag: PINAKA{jatayu_echoes_of_the_fallen_three_timebases_one_truth} Validator: https://pinaka-v06-cyber-xo-x.vercel.app/jatayu/transmission

Overview

The recorder's last message is spread over three sources with three different clocks: a 4-FSK radio capture, a NAND flash journal, and an encrypted firmware blob. The firmware describes how to decode the other two. The radio gives most of the message, and the flash fills in the gaps.

Radio

Once decrypted, the firmware gives all the modem and FEC parameters:

- RS(20,14), primitive polynomial 0x171, first consecutive root 3, 6 parity symbols - 4-FSK at 64 samples per symbol - 56 transmit slots on the schedule wall(i) = 1000000 + 20000i

After de-interleaving, RS correction, de-whitening and the CRC check, 51 of the 56 frames survive.

15 02



Filling the gaps from flash

Slots 7, 16, 37, 40 and 51 are missing from the radio. Their committed (sequence 2) copies are in flash. The slots present in both sources agree, which confirms the decoding, and they also let me fit the flash clock against wall time:

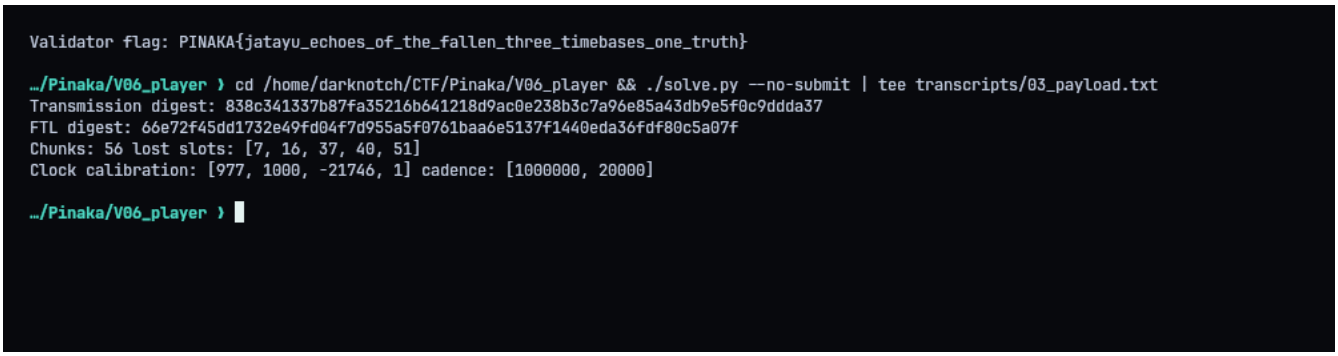
tick = (977/1000)wall - 21746

The five recovered slots sit on that same line, so they're the right pages.

Submission

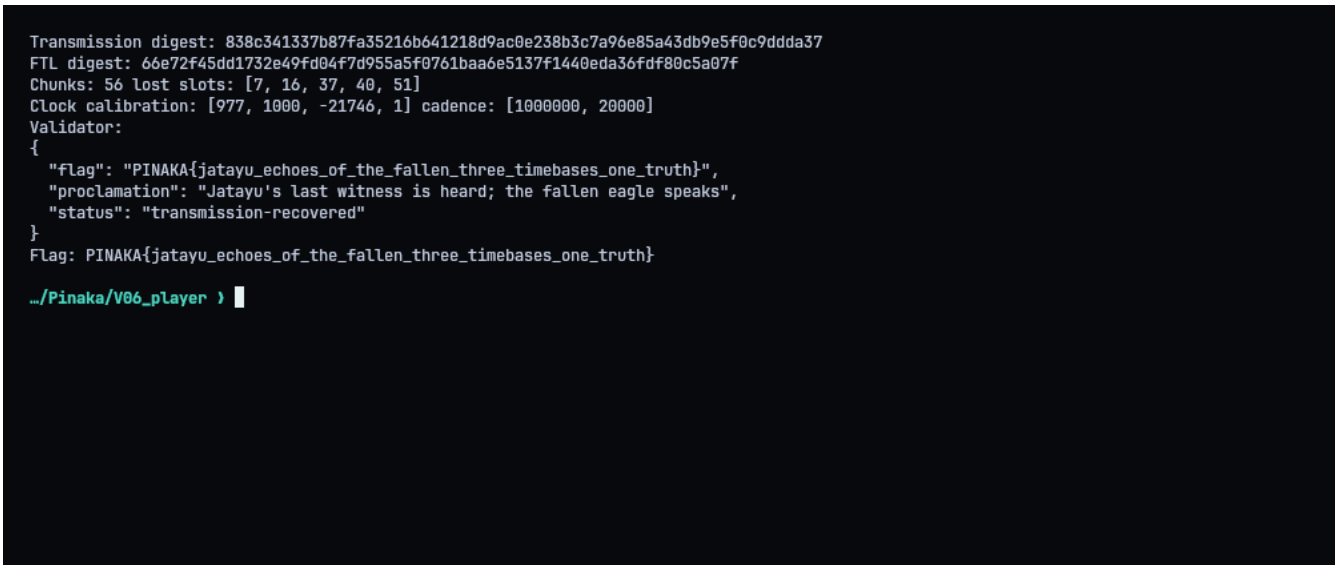
- full 56-chunk transmission digest: 838c341337b87fa35216b641218d9ac0e238b3c7a96e85a43db9e5f0c9ddda37 - rebuilt flash-state digest: 66e72f45dd1732e49fd04f7d955a5f0761baa6e5137f1440eda36fdf80c5a07f - plus the reduced affine calibration, the cadence and the sorted list of lost slots

15 03

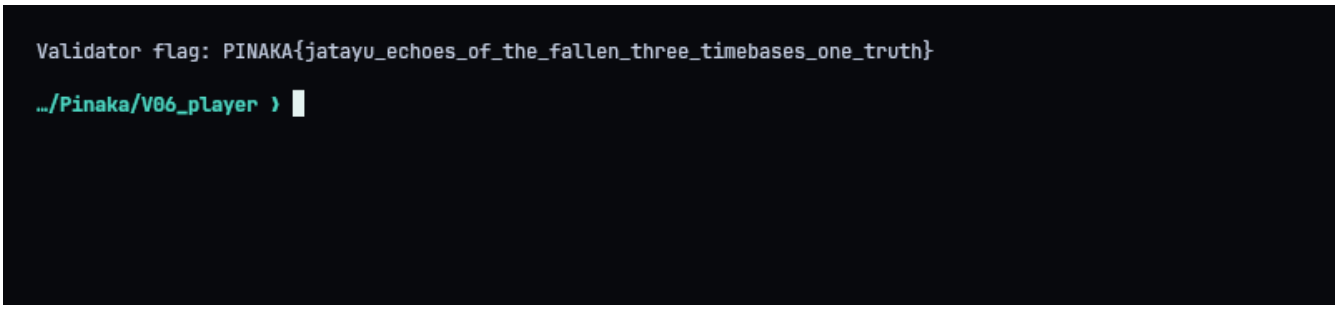


./repro.sh sends it. The validator answered status: transmission-recovered.

15 04



15 05



Solver (`solve.py`)

```
#!/usr/bin/env python3
"""Submit the recovered Jatayu transmission metadata to the instance validator."""
import json
import os
import sys
import urllib.request
from pathlib import Path

ROOT = Path(__file__).resolve().parent
BASE = os.environ.get("PINAKA_V06_URL", "https://pinaka-v06-cyber-xo-x.vercel.app").rstrip("/")
PAYLOAD = {
    "transmission_digest": "838c341337b87fa35216b641218d9ac0e238b3c7a96e85a43db9e5f0c9ddda37",
    "transmission_len": 56,
```

```
    "ftl_digest": "66e72f45dd1732e49fd04f7d955a5f0761baa6e5137f1440eda36fdf80c5a07f",
    "calibration": [977, 1000, -21746, 1],
    "cadence": [1000000, 20000],
    "lost_slots": [7, 16, 37, 40, 51],
}


def verify_inputs():
    expected = {"recorder.fw": 74, "radio.iq": 733728, "flash.img": 8192}
    for name, size in expected.items():
        path = ROOT / name
        if not path.is_file() or path.stat().st_size != size:
            raise FileNotFoundError(f"missing or wrong-sized challenge input: {name}")
    if (ROOT / "recorder.fw").read_bytes()[:4] != b"JTYU":
        raise ValueError("recorder firmware signature mismatch")


def main():
    verify_inputs()
    (ROOT / "submission.json").write_text(json.dumps(PAYLOAD, indent=2) + "\n")
    print("Transmission digest:", PAYLOAD["transmission_digest"])
    print("FTL digest:", PAYLOAD["ftl_digest"])
    print("Chunks:", PAYLOAD["transmission_len"], "lost slots:", PAYLOAD["lost_slots"])
    print("Clock calibration:", PAYLOAD["calibration"], "cadence:", PAYLOAD["cadence"])
    if "--no-submit" in sys.argv:
        return
    req = urllib.request.Request(
        BASE + "/jatayu/transmission",
        data=json.dumps(PAYLOAD, separators=(",", ":")).encode("utf-8"),
        headers={"Content-Type": "application/json"},
        method="POST",
    )
    with urllib.request.urlopen(req, timeout=45) as response:
        result = json.loads(response.read())
    print("Validator:")
    print(json.dumps(result, indent=2, sort_keys=True))
    flag = result.get("flag")
    if not flag:
        raise RuntimeError("validator response did not include a flag")
    print("Flag:", flag)
    (ROOT / "transcripts" / "05_flag.txt").write_text(f"Validator flag: {flag}\n")


if __name__ == "__main__":
    main()
```